



Linux Fundamentals

0%



Section 20 / 30

[Go to Questions](#) ↓

Working with Web Services

Another crucial element in web development is the communication between browsers and web servers. Setting up a web server on a Linux operating system can be done in several ways, with popular options including Nginx, IIS, and Apache. Among these, Apache is one of the most widely used web servers. Think of Apache as the engine that powers your website, ensuring smooth communication between your website and visitors.

We can also think of Apache like the foundation and framework of a house. Just as you can add different rooms or customize features in a house, Apache can be extended with modules, each designed for a specific purpose, whether it's securing communication, rerouting traffic, or dynamically shaping content like an interior designer rearranging rooms to fit your needs.

Apache's true strength lies in its modularity—it can be customized and extended with various modules to perform specific tasks. For example, `mod_ssl` acts like a lockbox, securing the communication between the browser and the web server by encrypting the data. The `mod_proxy` module is like a traffic controller, directing requests to the correct destination, especially useful when setting up proxy servers. Other modules such as `mod_headers` and

`mod_rewrite` give you fine control over the data traveling between browser and server, allowing you to modify HTTP headers and URLs on the fly, like adjusting the course of a stream.

In addition to handling static web content, Apache also supports the creation of dynamic web pages through server-side scripting languages. Commonly used languages include PHP, Perl, and Ruby, but you can also use others such as Python, JavaScript, Lua, or even .NET. These scripting languages serve as the creative tools behind the scenes, dynamically generating content and ensuring the website is interactive and responsive.

If you haven't already, let's install Apache:

```
shellsession
athane@htb[/htb]$ sudo apt install apache2 -y

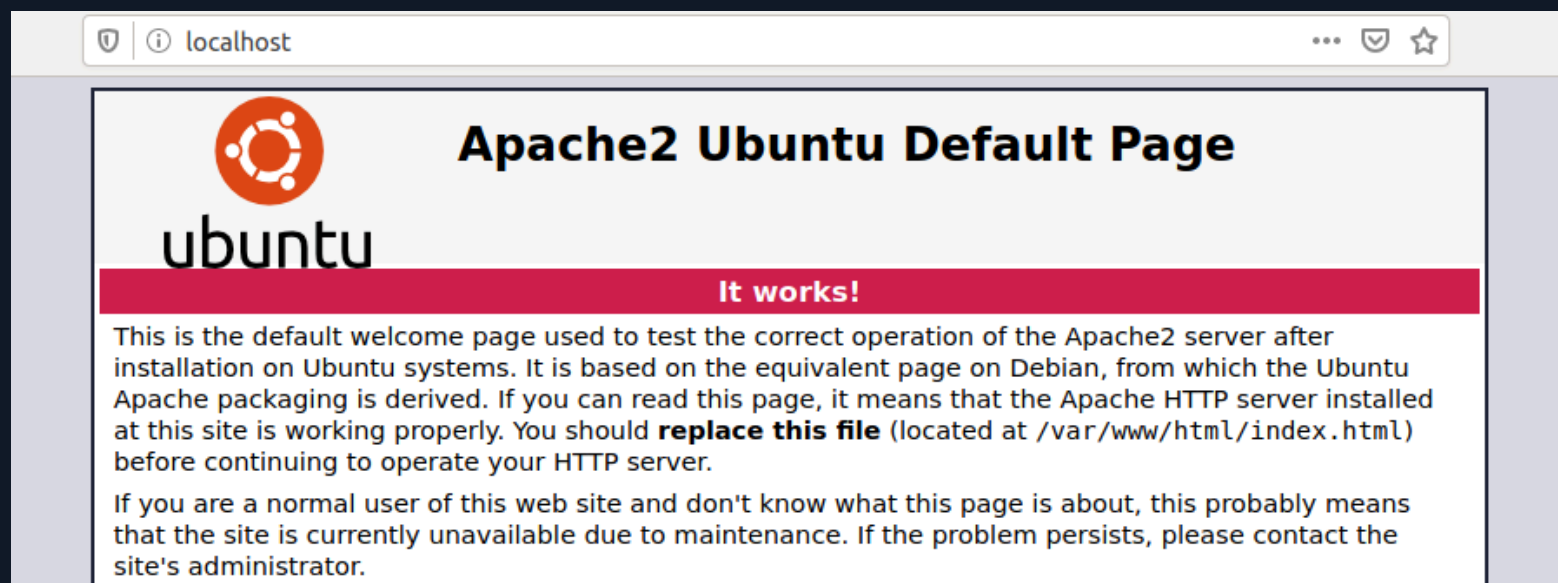
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom
The following NEW packages will be installed:
  apache2
0 upgraded, 1 newly installed, 0 to remove and 17 not upgraded.
Need to get 95,1 kB of archives.
After this operation, 535 kB of additional disk space will be used.
Get:1 http://de.archive.ubuntu.com/ubuntu bionic-updates/main amd64 apache2 amd64
2.4.29-1ubuntu4.13 [95,1 kB]
Fetched 95,1 kB in 0s (270 kB/s)
<SNIP>
```

Now, we can start the server using the `apache2ctl`, `systemctl` or `service` commands. There also exists an `apache2` binary, but it's generally not used to directly to start the server (this is due to the use of environment variables in the default configuration.)

shellsession

```
athane@htb[/htb]$ sudo systemctl start apache2
```

After Apache has been started, we navigate using our browser to the default page (<http://localhost>). By default, Apache will serve on HTTP port 80, and your browser will default to this port as well whenever you enter an HTTP **URI** (unless otherwise specified.)



This is the default page after installation and serves to confirm that the webserver is working correctly.

If you are using the Pwnbox, you might experience an error when attempting to start Apache; this is due to port 80 being occupied by another service. To set an alternate port for our web server, we can edit the `/etc/apache2/ports.conf` file. Here, we have set it to port 8080.

```
shellsession
GNU nano 2.9.3 /etc/apache2/ports.conf

# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 8080

<IfModule ssl_module>
Listen 443
</IfModule>

<IfModule mod_gnutls.c>
Listen 443
</IfModule>

^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos
M-U Undo
^X Exit          ^R Read File    ^\ Replace      ^U Uncut Text   ^T To Spell     ^_ Go To
Line  M-E Redo
```

We can now restart Apache and instead browse to `http://localhost:8080`, or could use a command line tool such as `curl` to verify:

```
athane@htb[/htb]$ curl -I http://localhost:8080
```

```
HTTP/1.1 200 OK
Date: Mon, 04 Nov 2024 21:18:50 GMT
Server: Apache/2.4.62 (Debian)
Last-Modified: Mon, 07 Oct 2024 06:39:39 GMT
ETag: "29cd-623dd48f6dd5a"
Accept-Ranges: bytes
Content-Length: 10701
Vary: Accept-Encoding
Content-Type: text/html
```

Another important aspect of working with web servers is learning how to communicate with them using command-line tools like curl and wget. These tools are incredibly useful when we want to systematically analyze the content of a webpage hosted on a web server. Think of them as your personal web browsers within the terminal, allowing you to fetch and interact with web content directly from the command line.

For example, we could write a simple bash script that downloads a webpage and extracts all the URLs it contains. This is akin to casting a net into a sea of information and pulling out the specific links we need. Such scripts are powerful for tasks like web scraping, automated testing, or monitoring website changes.

However, since our primary goal right now is to get familiar with Linux, you'll have the opportunity to see, build, and use such scripts in other modules. For now, let's focus on how we can interact

and communicate with a web server using curl and wget.

CURL

cURL is a tool that allows us to transfer files from the shell over protocols like **HTTP**, **HTTPS**, **FTP**, **SFTP**, **FTPS**, or **SCP**, and in general, gives us the possibility to control and test websites remotely via command line. Besides the remote servers' content, we can also view individual requests to look at the client's and server's communication. Usually, **cURL** is already installed on most Linux systems. This is another critical reason to familiarize ourselves with this tool, as it can make some processes much easier later on.

```
athane@htb[/htb]$ curl http://localhost
```

shellsession

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2016-11-16
    See: https://launchpad.net/bugs/1288690
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
...SNIP...
```

In the title tag, we can see that it is the same text as from our browser. This allows us to inspect the source code of the website and get information from it. More specifically, `curl` returns the website's page source as STDOUT. As opposed to viewing a website with a browser, which renders the HTML, CSS, and Javascript to create visual, aesthetic websites. Nevertheless, we will come back to this in another module.

Wget

An alternative to curl is the tool `wget`. With this tool, we can download files from FTP or HTTP servers directly from the terminal, and it serves as a solid download manager. If we use wget in the same way, the difference to curl is that the website content is downloaded and stored locally, as shown in the following example.

```
athane@htb[/htb]$ wget http://localhost
```

```
--2020-05-15 17:43:52-- http://localhost/  
Resolving localhost (localhost)... 127.0.0.1  
Connecting to localhost (localhost)|127.0.0.1|:80... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 10918 (11K) [text/html]  
Saving to: 'index.html'
```

```
index.html          100%[=====>]  10,66K  -  
.-KB/s      in 0s
```

```
2020-05-15 17:43:52 (33,0 MB/s) - 'index.html' saved [10918/10918]
```

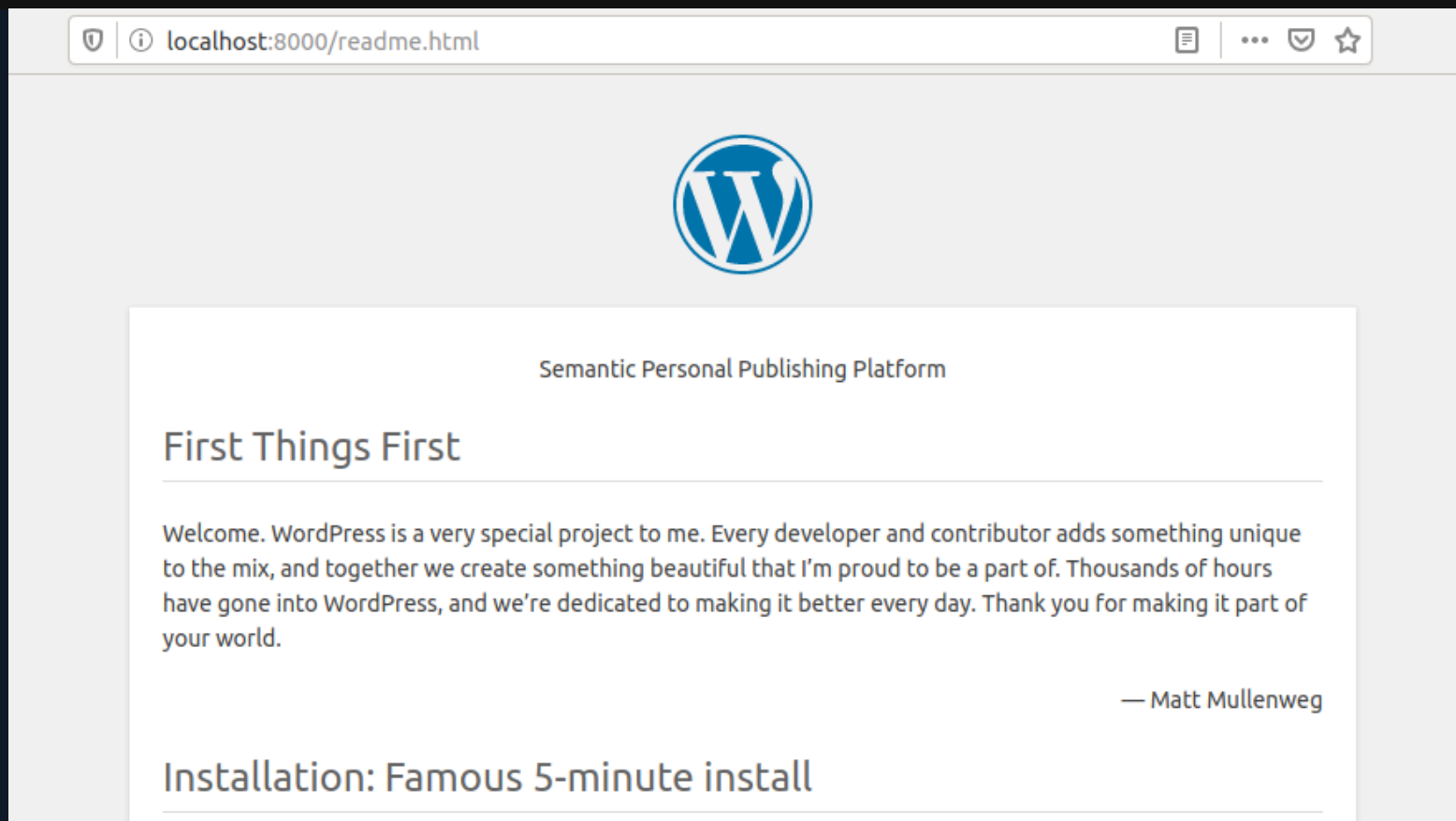
Python 3

Another option that is often used when it comes to data transfer is the use of Python 3. In this case, the web server's root directory is where the command is executed to start the server. For this example, we are in a directory where WordPress is installed and contains a "readme.html." Now, let us start the Python 3 web server and see if we can access it using the browser.

```
athane@htb[/htb]$ python3 -m http.server
```

shellsession

```
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

We can see what requests were made if we now look at our Python 3 web server's events.

```
shellsession  
athane@htb[/htb]$ python3 -m http.server  
  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...  
127.0.0.1 - - [15/May/2020 17:56:29] "GET /readme.html HTTP/1.1" 200 -  
127.0.0.1 - - [15/May/2020 17:56:29] "GET /wp-admin/css/install.css?ver=20100228  
HTTP/1.1" 200 -  
127.0.0.1 - - [15/May/2020 17:56:29] "GET /wp-admin/images/wordpress-logo.png  
HTTP/1.1" 200 -
```

```
127.0.0.1 - - [15/May/2020 17:56:29] "GET /wp-admin/images/wordpress-logo.svg?ver=20131107 HTTP/1.1" 200 -
```

In penetration testing, we often find ourselves facing challenges that require creative problem solving and out-of-the-box thinking. You'll encounter scenarios you haven't dealt with before, which means not only learning something new but also figuring out solutions on your own through research and innovative thinking.

Remember, this is a learning process, not an exam. Doing your own research and investigating different approaches is essential for expanding your skill set. In fact, these efforts will be key in building your expertise and adaptability in the field. From this point onward, the exercises you encounter will intentionally push you beyond your comfort zone. This is by design—to accelerate your learning and help you improve more quickly.

As you face these challenges, you'll develop the skills needed to tackle real-world situations, where there's often no one-size-fits-all solution. Embrace this process of exploration and discovery, as it's the best way to grow.

Connect to HTB



Enable step-by-step solutions ⓘ

PRO

Question 1



Find a way to start a simple HTTP server inside Pwnbox or your local VM using "npm". Submit the command that starts the web server on port 8080 (use the short argument to specify the port number).

 Submit

Hint

Question 2



Find a way to start a simple HTTP server inside Pwnbox or your local VM using "php". Submit the command that starts the web server on the localhost (127.0.0.1) on port 8080.

 Submit

20 / 30 Sections

